POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EICTA Third Project Delivery - MongoDB

Author(s): **Dario Antonino Santo (10733108)**
Caterina Giulia Scarpetta (11095784)
Matteo Bregola (11095785)
Emily Tiphareth Arcos (11021351)

Group Number: **5**

Academic Year: 2024-2025

Contents

Contents **i**

1 MongoDB **1**

1.1 ER to MongoDB Mapping 1

1.2 Database Schema 3

1.3 Database Schema details 3

1.4 Database Considerations 6

1.5 Queries 7

1.5.1 Creation/Update/Delete queries 7

1.5.2 Queries with projections and logical query operators 11

1.5.3 Queries with limit, sort, and comparison query operators 13

1.5.4 Query element query operators 16

1.5.5 Query with a simple aggregation 17

1.5.6 Query with an unwind and an aggregation 19

1.5.7 Query with an unwind, an aggregation, and followed by a filter . . . 20

1.5.8 Query with two conditions (evaluated separately) on a sub-document of an array 21

1.5.9 Query with two conditions (evaluated simultaneously) on a sub-document of an array 22

2 Accessing the MongoDB Database **25**

1 | MongoDB

1.1. ER to MongoDB Mapping

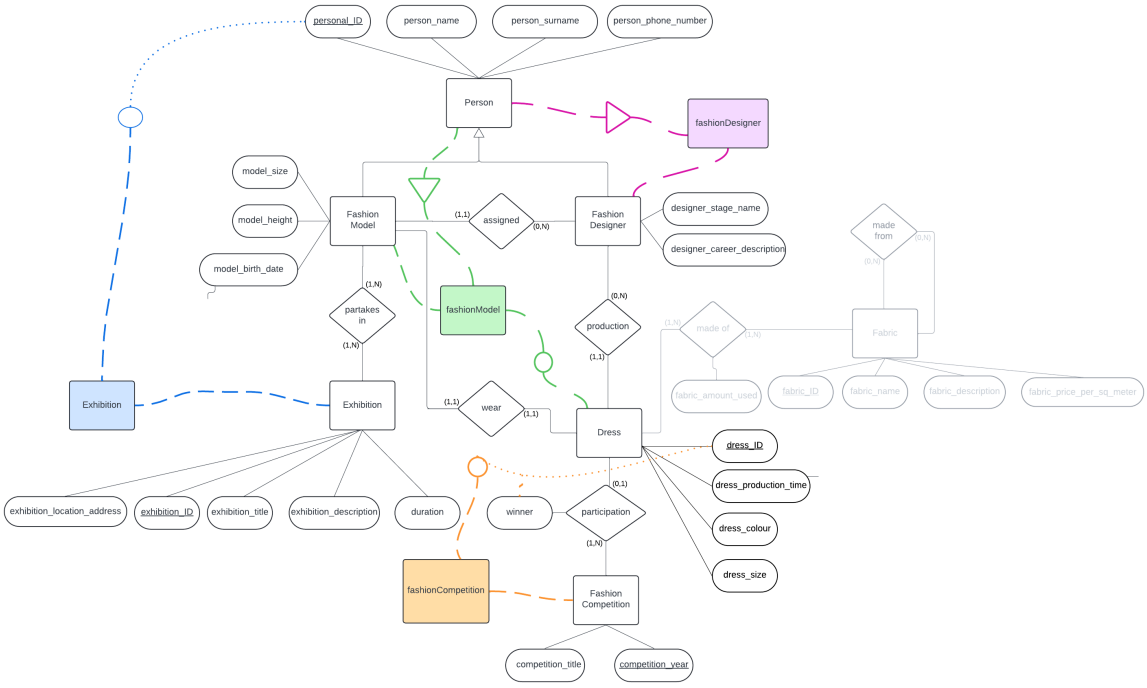

Figure 1.1: ER to MongoDB Mapping

We provide a visual representation on how we "mapped" the ER model to the MongoDB documents. The colored rectangles represents the documents, they are linked with two type of lines:

- **Dashed Lines**: represent that the new document comprehends all the attributes of the “old” ER entity as fields.
- **Dotted Lines**: represent that the new document comprehends the single attribute of the ‘old” ER entity as fields.

Furthermore, there are two other elements:

- **Circles**: represent that the new document comprehends the entity (or the attribute) as an array.
- **Triangle**: represent that the new document comprehends the entity (or the attribute) as a sub-document.

1.2. Database Schema

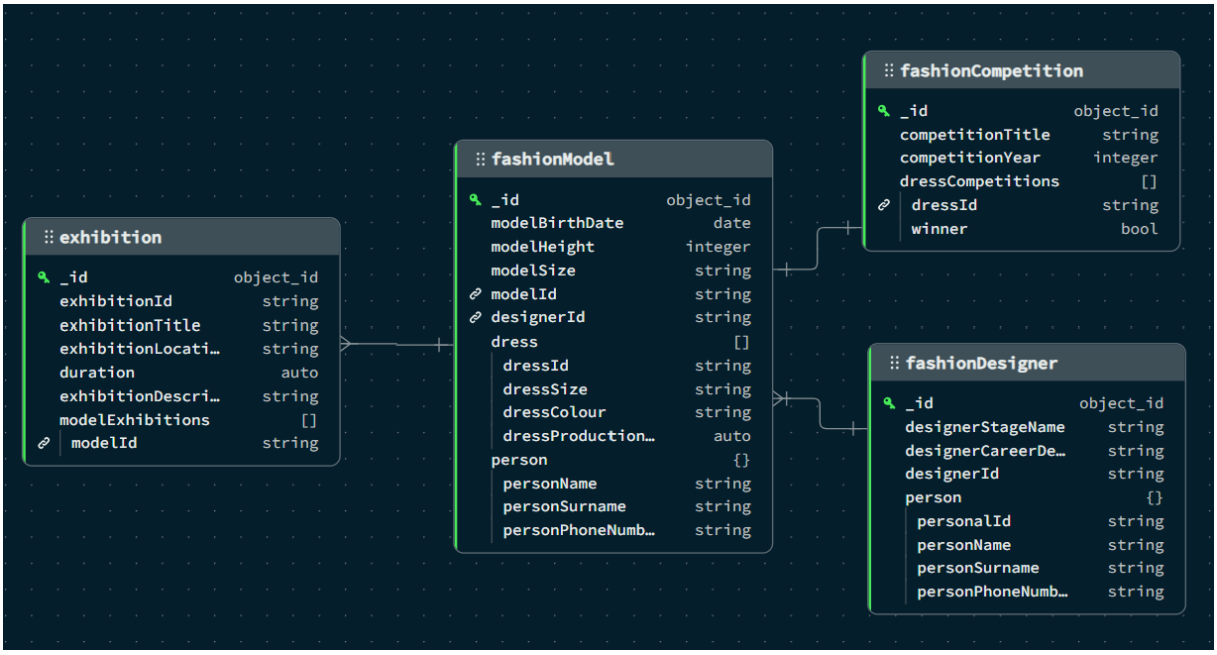

Figure 1.2: Mongodb Documents structure

1.3. Database Schema details

The entities we converted into nodes in our MongoDB database include:

- Person
- Fashion Model
- Fashion Designer
- Dress
- Exhibition
- Fashion Competition

Attributes: Next, following Project 1 and 2, we associated attributes to the entities identified based on the text descriptions below:

- Fashion Model: "Each model provides their personal data, including Personal ID, name, surname, birth date, phone number, height, and dress size (XS, S, M, L, XL, etc.)"

- Fashion Designer: "described by their Personal ID, name, surname, stage name , phone number, and a brief description of their career"
- Dress: "identified by a unique ID, size (XS, S, M, L, XL, etc.), colour, and production time"
- Fashion Exhibition: "described by a unique ID, duration (in minutes), title, location address, and description"
- Fashion Competition: "Once a year, a fashion competition is held". This entity did not have any direct listed attributes, but given this sentence in the text we decided to give this entity a title, as it would might be relevant.
- "Winner" in relationship "participation": the participation of a dress in a fashion competition came with the attribute "winner" to describe whether the dress had won the competition or not. This attribute was kept assigned in MongoDB as it connects each dress to the competition they participate in.

The attributes have been listed in the database schematic representation and their types follow from the previous SQL and Neo4J database.

Schematic Choices and Assumptions:

The below list defines the processes taken for this project to remain true to the original ER Model (cardinalities, relationships and attributes) AND correctly represent the entities using MongoDB's structural abilities.

- Person Entity:

- The *Person* entity served a purpose in the relational model for the ISA relationship between Fashion Model and Fashion Designer. Within MongoDB, these shared attributes between the model and designer can be instead represented as a subdocument with the 'Person' attributes: name, surname and phone number.

- Fashion Model Entity:

- *Project 1 Assumption on cardinality: A fashion model can be assigned to only one fashion designer but one fashion designer can be assigned to none or many models.* The one-to-many relationship between model and designer is represented by the presence of the 'designerId' in the Model document.
- *Project 1 Assumption on cardinality:A fashion model can only wear one dress in each*

fashion exhibition alongside the text cardinality description *Each dress is worn by one model.* From these and when looking at the ER model, we can emphasize that the system requires that one model can only wear one dress. Given this 1-to-1 structure, we decided to integrate as a sub-document the dress entity into the Fashion Model.

- From the above, and the *Project 1 Assumption on cardinality: A fashion designer can produce none or many dresses and one dress can only be made by one fashion designer* we concluded that if dress is now a sub-document within fashion model, and can be only made by one designer, then the designerId already referencing the designer in the Fashion Model document suffices to also match the dress to the same designer.

- Fashion Exhibition Entity:

- *Project 1 Assumption on cardinality: A fashion model can partake in one or many fashion exhibitions and one exhibition can have one or many models partaking in it.* From this assumption, we identify a many-to-many relationship between models and exhibitions - in order to deal with this, we decided to create a modelExhibition collection within the exhibition documents to track the list of models partaking in each exhibition. The collection contains the modelId and no other attributes because the collection references the fashionModel document and data about the model can be found from there.

- Fashion Competition Entity:

- The original text contains the following cardinality description: *Dresses can be submitted to one competition only* and we made the following *Project 1 Assumption: In the first project, we also made the assumption that the winning dress in a competition is assigned a dress and the competition it participated in.* From these, we identified a many-to-one relationship between dresses and competitions - in order to deal with this, we created a dressCompetitions array within the competition documents to track the list of dress IDs participating the competition AND the winning boolean for that dress.

1.4. Database Considerations

The conversion from mySQL to the MongoDB database led to the dress entity to be represented by an array inside fashionModel documents. As we said in the previous chapter however, we are working under the clear assumption that a fashion model has **one** dress uniquely assigned to them.

1.5. Queries

1.5.1. Creation/Update/Delete queries

01. Create a new exhibition document with 173626, with name In Vogue lasting 1h, set in Los Angeles with a short description and with models with ID 242129 and 406939 participating in it

```
db.exhibition.insertOne({ "exhibitionId": "173626",
"exhibitionTitle": "In Vogue",
"exhibitionLocation": "Los Angeles",
"Duration": 60,
"exhibitionDescription": "New trends of 2024",
"modelExhibitions": ["242129", "406939"]
} )
```

Query Description:
The query first points to the exhibition documents and uses the insertOne() method to list a new document and its given attributes. The modelExhibitions array is set up using square brackets to ensure the required structure is met.

Query Result:

```
Result
{
  acknowledged: true,
  insertedId: ObjectId('674c5c0b1baa136c0f839f0f')
}
```

02. Update the designer description for Fashion Designer ID "636448"

```
db.fashionDesigner.updateOne(
{ "designerId": "636448" },
{ $set:
{ "designerCareerDescription": "Timeless elegance and
   modern innovation" }
}
)
```

Query Description:
The query updates a specific document in the "fashionDesigner" where the "designerId" matches the ID "636448". We use the "$set" operator to update the "designerCareerDescription" field with the new value "Timeless elegance and modern innovation".
Query Result:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

03. Update the location address of an exhibition with the title 'Sydney Fashion Week' and a duration of 117 minutes to 'Sydney'

```
db.exhibition.updateOne(
{"$and":[
  {"exhibitionTitle": "Sydney Fashion Week"},
  {"duration": 117}
]},
  {"$set": {"exhibitionLocationAddress":"Sydney"}}
)
```

Query Description:
The query, using the updateOne statement will search for a document with an exhibitionTitle of "Sydney Fashion Week" and a duration of 117 minutes. The ''`$and`'' logical

operator ensures that both conditions are respected. Then the ‘‘$set’’ statement updates the value of “exhibitionLocationAddress” to “Sydney”.

Query Result:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

04. For a model with id 139716, update the dress colour they wear to taupe and their height to 169cm

```
db.fashionModel.updateOne(
{"modelId": "139716"
}, {
$set: {
        "dress.$[].dressColour": "Taupe",
        "modelHeight": 169}
})
```

Query Description:

The query firstly uses the updateOne() function over the fashionModel documents over the condition that the modelId is equal to 139716. Next, using the $set method, we update two values: the first is from the array dress within the model document with that id - where we retrieve the dressColour attribute and update it to "Taupe". The next value updated using the method is modelHeight which is set to 169 as the unit is already in cm.

Query Result:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

05. Delete all fashion competitions that occurred before the 80s

```
db.fashionCompetition.deleteMany( { "competitionYear":
    { "$lt": 1980 } )
```

Query Description:
This query uses the deleteMany() method to delete all fashionCompetition documents that have competitionYear less than 1980 (before 1980).

Query Result:

```
{
  acknowledged: true,
  deletedCount: 4
}
```

1.5.2. Queries with projections and logical query operators

01. Find Models of height 172 and size "M"

```
db.fashionModel.find({
    "$and":[
        {modelSize:"M"},
        {modelHeight:172}
        ]},
    {"person.personName":1, "modelSize":1, "
       modelHeight":1}
)
```

Query Description:
The query filters, using the ”and” logical query operator, models 172 (cm) high and wear a size “M”. Then it performs a projection, keeping only the name,size, and height.
Query Result:

```
{
  _id: ObjectId('674ef6cd6400e9791caeb718'),
  modelHeight: 172,
  modelSize: 'M',
  person: {
    personName: 'Kui'
  }
}
```

02. Return descriptions of exhibitions that last 49 minutes or are located in Australia

```
db.exhibition.find({
"$or":[
  {"exhibitionLocationAddress":"Australia"},
  {"Duration":49}
]}, {"exhibitionDescription":1})
```

Query Description:
The query will find inside the exhibition’s collection either exhibitions located in Australia

or having a duration equal to 49 minutes using the $or mehtod. This is done thanks to the `$or` operator. Finally with this statement `{"exhibitionDescription":1}` (called projection) the query will return only the exhibitions's descriptions.

Query Result:

```
{
  _id: ObjectId('674ef6cd6400e9791caeb6c1'),
  exhibitionDescription: 'Sustainable Style: Fashion
     with a Purpose'
}
{
  _id: ObjectId('674ef6cd6400e9791caeb6cf'),
  exhibitionDescription: 'Chic and Sleek: Effortlessly
      Stylish'
}
```

1.5.3. Queries with limit, sort, and comparison query operators

01. Return the earliest fashion competition titled 'Fashion Week' held after the 2000s

```
db.fashionCompetition.find(
  {
    "and": [
      {"competitionTitle": "Fashion Week"},
      {"competitionYear": { $gt: 2000 }}
    ]
  }
).sort({"competitionYear": 1}).limit(1)
```

Query Description:
The query operates within the fashionCompetition collection, searching for the first document where the competition title is equal to "Fashion Week" and the competition was held after the year 2000. The `$and` operator ensures both conditions are satisfied. The .sort("competitionYear": 1) sorts the results in ascending order of the competition year. Finally, the .limit(1) statement restricts the result to only the earliest match.
Query Result:

```
{
  _id: ObjectId('674ef6cd6400e9791caeb6ed'),
  competitionTitle: 'Fashion Week',
  competitionYear: 2011,
  dressCompetitions: [
    {
      winner: true,
      dressId: '841077'
    }
  ]
}
```

02. Retrieve fashionModels wearing a “Violet” dress in size “M”.

```
db.fashionModel.find(
  {
    "dress": {
      "$elemMatch": {
        "dressColour": "Violet",
        "dressSize": "M"
      }
    }
  }
).sort({dressID:-1}).limit(2)
```

Query Description:
This query matches the "fashionModel" and looks up in the documents where the dress array contains at least one object with the "dressColour" equal to "Violet" AND the "dressSize" equal to "M" (Medium) size. The element "elemMatch" is an operator to ensure both conditions are met in the same array. Lastly, we sort the results by descending order of the “dressID”, afterwards we limit the result to 2. **Query Result:**

```
{
  _id: ObjectId('674ef6cd6400e9791caeb720'),
  modelBirthDate: 1983-11-10T00:00:00.000Z,
  modelHeight: 199,
  modelSize: 'M',
  modelId: '260010',
  designerId: '246042',
  dress: [
    {
      dressId: '242985',
      dressSize: 'M',
      dressColour: 'Violet',
      dressProductionTime: 155
    }
  ],
  person: {
    personName: 'Ku',
```

```
    personPhoneNumber: '+420 212 197 3064',
    personSurname: 'Dadson'
  }
}
{
  _id: ObjectId('674ef6cd6400e9791caeb728'),
  modelBirthDate: 1991-02-28T00:00:00.000Z,
  modelHeight: 220,
  modelSize: 'M',
  modelId: '469547',
  designerId: '742814',
  dress: [
    {
      dressId: '357420',
      dressSize: 'M',
      dressColour: 'Violet',
      dressProductionTime: 147
    }
  ],
  person: {
    personName: 'Kalliste',
    personPhoneNumber: '+420 774 258 8902',
    personSurname: 'Gilhooly'
  }
}
```

1.5.4. Query element query operators

01. Return the youngest fashionModel

```
db.fashionModel.find({
"modelBirthDate": {"$type":"date"}
}).sort({"modelBirthDate":-1}).limit(1)
```

Query Description:

The query will search within the fashionModel collection using the "find" operator. It will look for fashionModel with an attribute named "modelBirthDate". The ''$type'' query operator ensures that this field is of type "date" and retrieves the matching documents. The "sort" operator organizes the results in descending order based on the "modelBirthDate" field, and the limit statement restricts the output to the first document.

Query Result:

```
{
  _id: ObjectId('674ef5ad6400e9791caeb6b6'),
  modelBirthDate: 2005-05-11T00:00:00.000Z,
  modelHeight: 190,
  modelSize: 'XXS',
  modelId: '961518',
  designerId: '401620',
  dress: [
    {
      dressId: '421933',
      dressSize: 'XXS',
      dressColour: 'Green',
      dressProductionTime: 173
    }
  ],
  person: {
    personName: 'Sayres',
    personPhoneNumber: '+66 715 178 8312',
    personSurname: 'Altham'
  }
}
```

1.5.5. Query with a simple aggregation

01. Return the number of clothes for each size

```
db.fashionModel.aggregate([
  {
    $group: {
      _id: "$modelSize",
      totalModels: { $sum: 1 }
    }
  },
  {
    $project: {
      size: "$_id",
      totalModels: 1,
      _id:0
    }
  }
]);
```

Query Description:
The query groups fashion models based on the *modelSize* and counts the number of models for each group. The projection renames the *_id* field as *size*, it includes *totalModels* as part of the output and explicitly removes the*_Id* that would otherwise have been included automatically.
Query Result:

```
{
  totalModels: 5,
  size: '2XL'
}
{
  totalModels: 4,
  size: '3XL'
}
{
  totalModels: 4,
  size: 'XL'
```

```
}
{
  totalModels: 7,
  size: 'S'
}
{
  totalModels: 6,
  size: '4XL'
}
{
  totalModels: 10,
  size: 'L'
}
{
  totalModels: 6,
  size: 'M'
}
{
  totalModels: 2,
  size: 'XXS'
}
{
  totalModels: 6,
  size: 'XS'
}
```

1.5.6. Query with an unwind and an aggregation

01. Return the fashion designers that have produced 3 or more dresses in sizes "L".

```
db.fashionModel.aggregate([
  { $unwind: "$dress" },
  { $match: { "dress.dressSize": "L" } },
  {
    $group: {
      _id: "$designerId",
      count: { $sum: 1 }
    }
  },
  { $match: { count: { $gte: 3 } } }
])
```

Query Description:
This query identifies fashion designers who have produced 3 or more dresses in a Large size. We start by unwinding the dress array, making it possible to separate the documents for each one of the dresses. Then, we match the dress sizes to only include those ones with a size of "L". Afterwards, we group the results by the designerID in order to count the number of the "L" size dresses produced by each one. Finally, we filter again to include those designers whose count of "L" dresses is greater than or equal to 3.
Query Result:

```
{
  _id: '844141',
  count: 3
}
```

1.5.7. Query with an unwind, an aggregation, and followed by a filter

01. Return the dress sizes that have exactly 5 dresses made in that size

```
db.fashionModel.aggregate([
                {"$unwind": {"path":"$dress"}},
{
        "$group": {
                "_id": "$dress.dressSize",
                "dresses_per_size":{"$sum": 1}
}
},
{"$match": {"dresses_per_size":{"$eq":5}}},
{"$project": {"dresses_per_size":0}
}
])
```

Query Description:
The query firstly uses the aggregate method over the fashionModel documents to allow for an aggregated outcome once the query is run. Within this method, the unwind stage is used to retrieve the content of the dress array and create new documents for each document in the array. Next, the group stage groups together the documents by their dress sizes and sums how many documents there are per size.

Using the match and project stages allows us to add a filter over the dresses_per_size aggregate and project the outcome. The match stage takes the dresses_per_size sums and filters over only the size value for which there are 5 dresses made in that size. Projecting the outcome will return a collection of sizes satisfying this filter.

Query Result:

```
{
  _id: '2XL'
}
```

1.5.8. Query with two conditions (evaluated separately) on a sub-document of an array

01. Find fashion competitions in which participated the dress with id=242985 or in which someone won

```
db.fashionCompetition.find({
    "dressCompetitions.winner":  true,
    "dressCompetitions.dressId": "242985"}
)
```

Query Description:
The query searches for fashionCompetitions documents were the dressCompetition array contains at least one element (dress) with winner set to true (dress that have won the competitio) and a dress(that can be the same one or not) which has dress id="24298".

Note 1: the first condition applied in this query doesn't filter out any document since every competition has one winner, however there weren't many interesting possibilities for creativity to fulfill this request, given our schema.
Note 2; see next query's note.

Query Result:

```
{
  _id: ObjectId('674ef6cd6400e9791caeb6de'),
  competitionTitle: 'iHeartRadio Music Awards Red
     Carpet',
  competitionYear: 1982,
  dressCompetitions: [
    {
      winner: false,
      dressId: '242985'
    },
    {
      winner: true,
      dressId: '496003'
    },
    {
```

```
      winner: false,
      dressId: '825876'
    }
  ]
}
```

1.5.9. Query with two conditions (evaluated simultaneously) on a sub-document of an array

01. Find fashion competitions in which a dress with ID 242985 participated and won

```
db.fashionCompetition.find({
  dressCompetitions:{
        $elemMatch:{
      "winner": true,
      "dressId": "242985"}
      }
})
```

Query Description:
The query searches into the *fashion competition* collection for documents in which there exists at least one document in the *dress competition* array which satisfies the conditions specified by the *$elemMatch*, The conditions specified on the dressCompetition document are:

- winner=true and
- dressid=242985

Note: The difference between this first query and the previous one is that the previous one applied the 2 conditions to possibly different documents of the array while this one searched for a document satisfying both. We interpreted the concept of separate and simultaneous in this way:

- Separate: condition evaluation process is carried on for the whole "scrolling" of the subdocuments, therefore we need to find fields of any document matching the specified conditions.
- Simulatenous: we evaluate each document, the document is ok if it has simultane-

ously both conditions, otherwise we go to the next one “forgetting” any info on the previous.

Query Result:

no result

2 | Accessing the MongoDB Database

As part of our Project delivery, we have added a database dump in the submitted folder with MongoDB files. However, for ease of connection we also wanted to provide access to the **MongoDB Compass** database we created:

URI: mongodb+srv://dario:dario6699@cluster0.hlnuz.mongodb.net/
Database: finaldelivery

The database shared consists of the documents created with the MongoDB structure **before** running any of the Create/Update/Delete queries required for the delivery.